

Finding the longest common subsequence for multiple biological sequences by ant colony optimization[☆]

Shyong Jian Shyu^{a,*}, Chun-Yuan Tsai^b

^aDepartment of Computer Science, Taipei Municipal University of Education, 1, Ai-Guo W. Road, Taipei 100, Taiwan, ROC

^bDepartment of Computer Science and Information Engineering, Ming Chuan University, 5 De Ming Road, Gwei Shan, Taoyuan 333, Taiwan, ROC

Available online 25 July 2007

Abstract

Finding the longest common subsequence (LCS) for a set of n (an arbitrary $n > 2$) sequences is an $\mathcal{NP}$ -hard problem. It is an essential operation for a wide range of applications in the areas of computational biology, pattern recognition, string editing and data compression, to name a few. In this paper, we design a novel ant colony optimization (ACO) algorithm to find the approximate solution to the LCS problem for multiple biological sequences. The performances of our ACO algorithm, the known expansion algorithm [Bonizzoni P, Vedova GD, Mauri G. Experimenting an approximation algorithm for the LCS. *Discrete Applied Mathematics* 2001;110:13–24] and best next for maximal available symbol algorithm [Huang KS, Yang CB, Tseng KT. Fast algorithms for finding the common subsequence of multiple sequences. In: *Proceedings of international computer symposium*; 2004. p. 90–95] were tested and compared by using various sets of DNA and protein sequences. The experimental results demonstrate the effectiveness and efficiency of the proposed ACO algorithm in dealing with the LCS problem for multiple biological sequences.

© 2007 Elsevier Ltd. All rights reserved.

Keywords: Longest common subsequence; Ant colony optimization; Meta-heuristics; Approximation algorithm; Biological sequence

1. Introduction

Finding the *longest common subsequence* (LCS, for short) for a set of multiple strings/sequences is an essential operation for a wide range of applications in the areas of computational biology, pattern recognition, string editing and data compression, to name a few [1–8]. Our interest in this paper is to find the LCS for a set of n ($n > 2$) biological sequences. In the sense of biology, a high sequence similarity usually implies the significant functional or structural similarity in biomolecular sequences (DNA, RNA or amino acid sequences) [7,8]. Evolution reuses, duplicates, builds on successful structures (proteins, exons, DNA regulatory sequences, morphological features, enzymatic pathways and so on). The same and related molecular structures show up repeatedly in the genome of a single species and across a very wide spectrum of divergent species. An LCS is therefore significant for the identification of maximally homologous subsequences among the species considered [3].

The LCS of a set of biological sequences usually preserves those informative segments (subsequences) that are conserved during the evolutionary process on condition that no internal deletions or insertions within contiguous segments

[☆] This research was partially supported by the National Science Council of Taiwan, Republic of China, under Grant NSC 93-2213-E-130-008.

* Corresponding author. Tel.: +886 2 23113040x8363; fax: +886 2 23112773.

E-mail addresses: sjshyu@tmue.edu.tw, sjshyu@mcu.edu.tw (S.J. Shyu).

would happen [3,7]. It is a fundamental basis for further molecular sequence analyses such as the multiple sequence alignment (where internal deletions or insertions within segments may happen), similarity-based gene prediction, genome rearrangement and so on [1,4,7–10].

The LCS problem for two sequences (2LCS, for short) can be solved in $O(mn)$ by using the dynamic programming technique where m and n are the lengths of the two input sequences. Various kinds of polynomial-time algorithms for the 2LCS problem have been proposed in the literature [1–3,11–15]. Yet, in the general case of $n(>2)$ sequences of length l each, the LCS problem is $\mathcal{NP}$ -hard even on a binary alphabet [16,17]. Some researchers proposed algorithms to solve the LCS problem with $n \geq 3$ sequences based upon dynamic programming [18,19], but they did not substantially improve the $O(l^n)$ time complexity. Such approaches are not applicable even for a small value of n in most practical situations, since the sequence length l is usually very large. In the cases of biological sequence analysis or text editing, l might be larger than 500. Therefore researchers have switched their interests to find heuristic algorithms in order to find approximate solutions to the LCS problem.

Some approximation algorithms for the LCS problem, such as the *long run* (LR) algorithm [20], *expansion algorithm* (ExpA) [21] and *best next for maximal available symbols* (BNMAS) algorithm [22], have been proposed with a guaranteed performance ratio (i.e. the ratio of the length of the optimal solution over that of the approximation) of $|\Sigma|$ that is the size of alphabet Σ of the input sequences (for DNA, $\Sigma_{\text{DNA}} = \{a, g, c, t\}$, $|\Sigma_{\text{DNA}}| = 4$; for protein, $|\Sigma_{\text{protein}}| = 20$). We shall give a brief review on these algorithms in Section 2.2. Even though they are significant in theoretical interests, they may not produce satisfactory solutions. An approximate solution with a performance ratio of 4 for DNA sequences (or 20 for protein sequences) is not attractive at all in practical concerns. There is still a strong demand to recruit LCS algorithms which are able to not only produce approximate solutions with good qualities but also accomplish the computational efficiency for real world applications. Meta-heuristics have exhibited many successful applications in solving combinatorial problems [23–25] in terms of solution quality and computational efficiency. Our aim in this paper focuses upon applying *ant colony optimization* (ACO) to resolve the LCS problem.

Ant colony algorithms were inspired by observations on the behavior of real ant colonies in foraging food. Such behavior is coordinated by *pheromone* which ants deposit and sense. While traveling back and forth from their colony to the food source, ants deposit pheromone on the ground forming pheromone trails. When deciding a path to follow, ants sense pheromone and tend to choose the ones with stronger pheromone intensities. As time passes, shorter paths accumulate more pheromone intensities than longer ones. Consequently this pheromone-trail-following behavior forms a positive feedback mechanism so that a colony of ants is able to exploit the pheromone trails formed by individual ants to discover the shortest path between the nest and the food source in a cooperative way.

Dorigo et al. [26,27] deliberately capsulated the positive feedback mechanism of the ant colonies in foraging food into a multi-agent approach, namely ACO. ACO has been successfully applied to resolve a wide range of combinatorial problems such as the traveling salesman problem, quadratic assignment problem, vehicle routing problem or problems in scheduling and telecommunications and so on. Please refer to [26–35] for a comprehensive understanding. Fig. 1 presents the outline of ACO meta-heuristics (a modified version from [30,35]).

As seen in Fig. 1, each artificial ant's solution-construction process in one *iteration* can be described by a sequence of transitions from a certain initial state to the target state (which represents a solution to the problem). In real implementation, each artificial ant has a local memory $\mathcal{M}$ to memorize the visited states in the current iteration and it would be reset in the beginning of each iteration. A function $\mathcal{A}$, which is composed by the locally available pheromone trail (reflecting the experience of previous ants in solution finding) and heuristic value (evaluating the preference of visiting next state in a local view) with respect to the ant/state, defines the *state-transition rule* that determines the set $\mathcal{P}$ of probabilities of all possible state-transitions from the current state subject to the problem constraints. By applying a certain decision policy based upon $\mathcal{P}$ and the problem constraints, the ant moves from state to state to construct its own solution. Essentially, in one iteration each ant incrementally constructs its solution by relying upon $\mathcal{A}$, $\mathcal{M}$, $\mathcal{P}$ which guide the ant towards the promising regions in the searching space stochastically as iterations grow. When each ant in the colony has constructed its own solutions (a situation referred to as a *generation*), the pheromone trails would be updated to reflect the learning experience about solution construction from previous ants. The optional *daemon_actions* can be adopted to perform some centralized actions such as carrying out a *local search* procedure to improve the quality of the solution or other operations from a non-local perspective.

It is our goal to design an effective and efficient ACO algorithm for the LCS problem, particularly for multiple and long biological sequences. The rest of the paper is organized as follows. We give a formal definition to the LCS problem in Section 2.1 and briefly introduce the three known (LR [20], ExpA [21] and BNMAS [22]) approximation algorithms

```

procedure ACO_metaheuristic()
  initialize_pheromone_and_ant_memory();
  while (termination criterion not satisfied) do
    for (each ant in the colony) do
       $\mathcal{M}$  = reset_ant_memory();
      while ( state  $\neq$  target_state )
         $\mathcal{A}$  = get_pheromone_and_heuristic_value();
         $\mathcal{P}$  = compute_transition_probabilities( $\mathcal{A}$ ,  $\mathcal{M}$ , problem_constraints);
        state = apply_ant_decision_policy( $\mathcal{P}$ , problem_constraints);
         $\mathcal{M}$  = update_ant_memory()
      endwhile // end of an iteration for each ant
    endfor // end of a generation for the colony
    pheromone_update();
    daemon_actions(); {optional}
  endwhile // end of the searching process
endprocedure

```

Fig. 1. Outline of the ACO meta-heuristic.

for the LCS problem in Section 2.2. The detailed design of the ACO algorithm for LCS is presented in Section 3. The experimental results of our ACO, the ExpA and BNMAS algorithms for a variety of biological sequences are compared and discussed in Section 4. Section 5 gives some concluding remarks.

2. Problem formulation of LCS and its approximation algorithms

2.1. LCS problem

Consider two sequences $s = s_1 s_2 \dots s_p$ and $t = t_1 t_2 \dots t_q$ over a finite alphabet Σ . s is a *subsequence* of t , denoted by $s < t$, if there exists an ordered set of p integers $(i_1, i_2, \dots, i_p)$, $1 \leq i_1 < i_2 < \dots < i_p \leq q$, such that $s[j] = t[i_j]$ where $1 \leq j \leq p \leq q$. Or equivalently, we may say that $s < t$ if s can be obtained by deleting some characters from t without disturbing the relative positions of the remaining elements. For instance, when $s = abc$ and $t = aabbccdd$ ($p = |s| = 3$ and $q = |t| = 6$ where $|s|$ denotes the length of s), we have $s < t$; since there exists $(i_1, i_2, i_3) = (1, 3, 5)$, $i_1 < i_2 < i_3$, such that $s[j] = t[i_j]$ for $1 \leq j \leq p (=3) \leq q (=6)$, i.e. $s[1] = t[i_1] = t[1] = a$, $s[2] = t[i_2] = t[3] = b$, $s[3] = t[i_3] = t[5] = c$.

Given a finite set of n sequences $S = \{s_1, s_2, \dots, s_n\}$ over a finite alphabet Σ , a *common subsequence* of S is a sequence c over Σ such that c is a subsequence of each sequence in S , i.e. $c < s_i$ where $s_i \in S$ for $1 \leq i \leq n$. The *LCS* of S is the common subsequence of the maximum length. Formally, the LCS problem with respect to S is to

$$\begin{aligned} & \text{maximize} && |c| \\ & \text{subject to} && c < s_i \quad \text{for } 1 \leq i \leq n, \end{aligned}$$

where $|c|$ denotes the length of c and $s_i \in S$.

We give a simple example for the LCS problem. Assume that $S = \{babacd, acdcbc, adcbc\}$ where $\Sigma = \{a, b, c\}$. It is easy to see that a, b, c, d are common subsequences of S with length 1; ab, ac, ad, bc are common subsequences of S with length 2; and abc is a common subsequence of S with length 3. There is no common subsequence of S with length larger than 3. Thus, abc is the LCS of S . Note that there may be more than one LCS for a set of sequences and all algorithms mentioned in this paper report only one.

2.2. Approximation algorithms for LCS

2.2.1. LR algorithm

The $\mathcal{NP}$ -hardness of the LCS problem was shown in [16,17]. Jiang and Li [20] also proved that there exists a constant $\delta > 0$ such that, if the LCS problem has a polynomial-time approximation algorithm with performance ratio n^δ , then $P = NP$. Despite the discouraging result, they proposed a simple greedy polynomial LR algorithm which finds

```

Expand( $S, e$ )
input: a set of  $n$  sequences  $S = \{s_1, s_2, \dots, s_n\}$  over  $\Sigma$  and a stream  $e = a_1 a_2 \dots a_k$  of  $S$ 
output: a common subsequence of  $S$  expanded from  $e$ 
for (  $1 \leq l \leq k$  ) do
     $j_l = 1$ 
endfor
repeat
    test = false
    for (  $1 \leq l \leq k$  ) do
        if (  $a_1^{j_1} a_2^{j_2} \dots a_l^{2j_l} \dots a_k^{j_k}$  is a subsequence of  $S$  ) then
             $j_l = 2 \times j_l$ 
            test = true
        endif
    endfor
until ( test = false )
for (  $1 \leq l \leq k$  ) do
     $j_l = \max \{ \alpha \}$  such that  $a_1^{j_1} a_2^{j_2} \dots a_l^\alpha \dots a_k^{j_k}$  is a common subsequence of  $S$ 
endfor
return(  $a_1^{j_1} a_2^{j_2} \dots a_k^{j_k}$  )

```

Fig. 2. Procedure $Expand(S, e)$.

an approximate solution to LCS with performance ratio $\sigma(=|\Sigma|)$. The LR algorithm finds a maximum t such that a^t is a common subsequence of all input sequences for some $a \in \Sigma$ and reports a^t as the approximation solution to LCS. Even though this algorithm produces an approximation solution with a guaranteed performance ratio to the optimum, it does not perform well in instances even with a small number of long sequences (say $n \leq 20$ and the length is larger than 100) [21]. In particular, the major weakness of the algorithm is that it reports a solution with only one symbol. Such a common subsequence is insignificant. For instance, it neither provides a strong enough evidence for being the crucial common signal in speech recognition, nor discovers any biological relevance about conserved regions in multiple sequence comparison.

2.2.2. ExpA algorithm

The ExpA algorithm for the LCS problem was proposed by Bonizzoni et al. [21]. Its performance ratio is σ too. The computational results obtained by ExpA are much better than those by LR in terms of solution quality [21] when random sequences or sequences generated by the Jukes–Cantor model of evolution [36] were tested. We describe ExpA briefly in the following.

Let a^j denote a string composed of j consecutive a 's where $a \in \Sigma$ and $j > 0$. A *basic sequence* of length k over Σ is a sequence $a_1 a_2 \dots a_k$ with $a_i \in \Sigma$ for $1 \leq i \leq k$ and $a_t \neq a_{t+1}$ for $1 \leq t < k$. Consider a set of n sequences $S = \{s_1, s_2, \dots, s_n\}$ over an alphabet Σ . e is a *stream* of S if e is a basic sequence over Σ and $e < s_i$ for $1 \leq i \leq n$. Given a sequence s over Σ , we can represent it as $s = a_1^{j_1} a_2^{j_2} \dots a_k^{j_k}$, where $a_i \in \Sigma$ and $j_i > 0$ for $1 \leq i \leq k$ and $a_t \neq a_{t+1}$ for $1 \leq t < k$ such that $a_1 a_2 \dots a_k$ is a stream of $\{s\}$ (note that a stream is defined with respect to a set of sequences) and $a_i^{j_i}$ is called the i th *block* of s . Let $e = a_1 a_2 \dots a_k$ be a stream of S . To obtain a common subsequence of S as long as possible based upon e , we can find $e' = a_1^{j_1} a_2^{j_2} \dots a_k^{j_k}$ by maximizing j_l for $1 \leq l \leq k$ such that $e' < s_i$ for $1 \leq i \leq n$ in which e' is called the *expand* of stream e in S .

Bonizzoni et al. thus developed a procedure to expand out a common subsequence of S from a stream e , referred to as $Expand(S, e)$ [21]. Basically $Expand(S, e)$ searches for a new common subsequence of S by scanning e from left to right and block by block iteratively and doubling the length of the examined block each time until no block of the expanded common sequence can be doubled. Then, it examines the expanded common sequence again from left to right to try to enlarge the size of each block in order to obtain a common subsequence as long as possible. Fig. 2 illustrates procedure $Expand(S, e)$.

```

Greedy( $S$ )
input: a set of  $n$  sequences  $S = \{s_1, s_2, \dots, s_n\}$  over  $\Sigma$ 
output: a long enough stream of  $S$ 
if ( $S$  contains only one sequence) then
    return(  $S$  )
endif
if ( $|S| = 2$ ) then
     $lcs = LCS(s_1, s_2)$ 
    return( the longest stream of  $lcs$  )
endif
for ( each  $s_i \in S$  ) do
    Replace  $s_i$  with the longest stream of  $\{s_i\}$ 
endfor
for ( each  $i, j$  such that  $1 \leq i < j \leq n$  ) do
     $s_{i,j} = |Greedy(\{s_i, s_j\})|$ 
     $temp[i, j] = |s_{i,j}|$ 
endfor
let  $i, j$  be the pair of indices that maximizes  $temp[i, j]$ 
return( $Greedy(S - \{s_i, s_j\} \cup s_{i,j})$ )

ExpA( $S$ )
input: a set of  $n$  sequences  $S = \{s_1, s_2, \dots, s_n\}$  over  $\Sigma$ 
output: an approximate longest common subsequence of  $S$ 
 $T = \{s : s \text{ is a stream of } S \text{ of length } \leq 2\}$ 
add to  $T$  all substrings of  $Greedy(S)$ 
initialize  $cs$  as an empty word
for ( each  $e \in T$  ) do
     $w = Expand(S, e)$ 
    if (  $w$  is longer than  $cs$  ) then
         $cs = w$ 
    endif
endfor
return(  $cs$  )

```

Fig. 3. Procedure *Greedy*(S) and expansion algorithm ExpA(S).

The essential ideas of ExpA are to find a target set T which contains streams of S and then to apply $Expand(S, e)$ procedure to expand each stream $e \in T$. Among the results, which are all common subsequences of S , the one with the maximum length is the solution reported by ExpA. Since the number of all streams over a binary alphabet is polynomial in the size of S , they simply generate the set of all streams of S to be T . If the input alphabet is more than binary, the number of all streams becomes exponential. Thus they conduct a heuristic approach to choose a subset of all streams as T in order to avoid the exponential time-complexity for producing all streams.

We discuss the case of using an arbitrary alphabet (not binary). They implemented their heuristic as procedure *Greedy*(S) which obtains a long enough stream st of S based upon the optimal 2LCS computations (denoted by function $LCS(s_1, s_2)$ which returns the optimal LCS of s_1 and s_2). Then T is defined to be all substring of st (which are all streams of S) and all streams of S of maximum length 2. Each element $e \in T$ is expanded by calling $Expand(S, e)$. The outcome with the maximum length is reported as the solution. Fig. 3 presents procedure *Greedy*(S) and expansion algorithm ExpA(S) [21].

The ExpA algorithm runs in $O(nl^4 \log l)$ time where n is the number of sequences and l is the length of the longest sequence. For a simple instance $S = \{a^4b^3a^4b^2a, a^3b^4a^4b^3\}$, the result by ExpA is $a^3b^3a^4b^2$, while that by LR is merely a^7 .

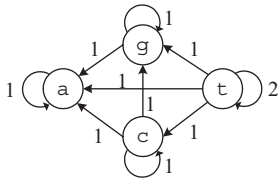
Bonizzoni et al. [21] tested ExpA and LR by using two sets of sequences: (1) four random (uniform distribution) sequences of length between 90 and 100, and (2) 5, 10 and 20 DNA or protein sequences generated according to the Jukes–Cantor model [36] with lengths ranging from 400 to 500, respectively. The computational results reveal that ExpA finds much longer common subsequences than LR for the LCS problems tested.

2.2.3. BNMAS algorithm

Huang et al. [22] proposed an approximation algorithm called BNMAS for the LCS problem. We summarize the ideas in BNMAS as follows. Consider a set of sequences $S = \{s_1, s_2, \dots, s_n\}$ over Σ . Let $\mathcal{V}_s(i, j)$ denote the number of character j prior to the last occurrence of i (including i) in sequence s and $\mathcal{V}(i, j) = \min_{s \in S} \{\mathcal{V}_s(i, j)\}$ which is referred to as the *vector of maximal available symbols* where $i, j \in \Sigma$. Consider a simple example of three sequences: $s_1 = \text{agctt}$, $s_2 = \text{tagact}$ and $s_3 = \text{cagtc}$ where $\Sigma = \{a, g, c, t\}$. The vectors of $\mathcal{V}_1(i, j)$, $\mathcal{V}_2(i, j)$, $\mathcal{V}_3(i, j)$ and $\mathcal{V}(i, j)$ for $i, j \in \Sigma$ are in the following:

	$\mathcal{V}_1$				$\mathcal{V}_2$				$\mathcal{V}_3$				$\mathcal{V}$			
	a	g	c	t	a	g	c	t	a	g	c	t	a	g	c	t
a	1	0	0	0	2	1	0	1	1	0	1	0	1	0	0	0
g	1	1	0	0	1	1	0	1	1	1	1	0	1	1	0	0
c	1	1	1	0	2	1	1	1	1	1	2	1	1	1	1	0
t	1	1	1	2	2	1	1	2	1	1	2	2	1	1	1	2

Let $d(j)$ denote the *degree* of j which is computed by $\sum_{i \neq j} \mathcal{V}(i, j)$. In fact, when we represent vector $\mathcal{V}$ as a weighted directed graph $G = (V, E)$ where $V = \Sigma$ and $\langle i, j \rangle \in E$ is an arc (from i to j) with weight $\mathcal{V}(i, j)$ for $\mathcal{V}(i, j) \neq 0$, $i, j \in V$, then $d(j)$ is the total weights of in-arcs from i ($\neq j$) to j . For the simple example above, the corresponding graph is:



where $d(a) = 3$, $d(g) = 2$, $d(c) = 1$ and $d(t) = 0$.

For $i, j \in V$ in the graph G corresponding to vector $\mathcal{V}$, i is called the *predecessor* of j if $V(i, j) > 0$. If there exists a directed path from g to h in G and g has no other predecessor but itself, then we call g the *most predecessor* of h , denoted by $g = p(h)$ where $g, h \in V$. For the above example, $p(a) = t$, $p(g) = t$, $p(c) = t$ and $p(t) = t$.

Assume that $t_1 t_2 \dots t_z$ is the common subsequence of S found by BNMAS. BNMAS constructs it by determining $t_z, t_{z-1}, \dots, t_2, t_1$ iteratively. Let t be the partially constructed solution. In the next iteration, BNMAS computes $\mathcal{V}(i, j)$, $d(j)$ and $p(j)$ ($i, j \in \Sigma$) for the sequences considered and finds out $g = p(h)$ as a new constituent to the solution, i.e. $t = g \| t$ (in which $\|$ denotes the concatenation operation of strings), where h is chosen under the heuristic that $d(h) + \max_{j \in \Sigma} \{\mathcal{V}(h, j)\}$ is maximal. Once g is found and t is updated, the sequences considered become $S - t = \{s_1 - p_1, s_2 - p_2, \dots, s_n - p_n\}$ where p_i is the suffix of s_i with a minimal length in the current iteration on condition that t is the common subsequence of p_i 's for $1 \leq i \leq n$. Initially t is set as null. When no more character g can be chosen from $S - t$ or some sequence in $S - t$ becomes null, BNMAS stops and t is reported as the solution.

We test BNMAS by using the above simple example: $S = \{\text{agctt}, \text{tagact}, \text{cagtc}\}$. Initially, $t = ""$. In iteration 1, $g = t$ ($= p(a)$, i.e. $h = a$) since $d(a) + \max_{j \in \Sigma} \{\mathcal{V}(a, j)\}$ is maximal so that $t = t$ and $S - t = \{\text{agct}, \text{tagac}, \text{cagtc}\}$. In iteration 2, $g = c$ ($= p(a)$) since $d(a) + \max_{j \in \Sigma} \{\mathcal{V}(a, j)\}$ is maximal (where $\mathcal{V}(i, j)$'s would be updated with respect


```

Bnmas( $S$ )
input: a set of  $n$  sequences  $S = \{s_1, s_2, \dots, s_n\}$  over  $\Sigma$ 
output: an approximate longest common subsequence of  $S$ 
 $t = \text{null}$ 
repeat
     $a = \text{LR}(S-t)$ 
    for ( $i, j \in \Sigma$ ) do
        calculate  $\mathcal{V}(i, j)$  based upon  $S-t$ 
    endfor
    for ( $j \in \Sigma$ ) do
        calculate  $d(j)$ 
    endfor
    find  $h$  with maximal  $(d(h) + \max_{j \in \Sigma} \{\mathcal{V}(h, j)\})$ 
    (If there are several choices, select  $h$  with  $d(h)$  is maximal and  $h = a$ ;
    otherwise, randomly select one with  $\max_{i \in \Sigma} (d(i) + \max_{j \in \Sigma} \{\mathcal{V}(i, j)\})$ )
    if ( $h \neq \text{null}$ ) then
         $t = p(h) \parallel t$ 
    endif
until ( $h = \text{null}$ )
return( $t$ )

```

Fig. 4. Procedure Bnmas(S).

to the corresponding $S-t$), and then $t = ct$ and $S-t = \{\text{ag}, \text{taga}, \text{cagt}\}$. In the subsequent iterations, g and a would be chosen and appended into the front of t , respectively, thus the final common subsequence is $t = \text{agct}$. If there are several choices for h , BNMAS selects the one with a maximal $d(h)$ and h is the same as the character obtained from the LR algorithm, or else it randomly picks up one.

The BNMAS algorithm is formally described in Fig. 4. Let $\bar{S}$ denote the set that consists of each of the reversed sequence in S . BNMAS reports the maximum of $\text{Bnmas}(S)$ and $\text{Bnmas}(\bar{S})$ as the final solution. The time complexity of BNMAS algorithm is $O(\sigma^2 nl + \sigma^3 l)$ where $\sigma = |\Sigma|$, n is the number of sequences and l is the length of the longest sequence in S . In the experiments reported in [22], BNMAS obtains better solutions than ExpA where four random sequences (with lengths 100, 300, 500 and 1000) over a binary alphabet were tested.

As mentioned earlier, the solution reported by LR contains only one symbol so that LR is hardly comparable to ExpA and BNMAS. Based upon the results of [22], BNMAS is a better approach than ExpA in producing a longer common subsequence for multiple sequences. However, the numbers of their test sequences as well as the alphabets were small (for ExpA against LR [21]: $n \leq 20$ and $|\Sigma| = 4$, while for BNMAS against ExpA [22]: $n = 4$ and $|\Sigma| = 2$). We still do not know the effectiveness and efficiency of ExpA and BNMAS when the problem size (n) and alphabet size ($|\Sigma|$) become large. Thus, the performances of the proposed ACO algorithm, ExpA and BNMAS for biological sequence sets with larger problem and alphabet sizes would be compared in Section 4.

3. ACO algorithm for LCS

Consider a set of n sequence $S = \{s_1, s_2, \dots, s_n\}$ over an alphabet Σ . Let $s_i[j]$ denote the j th character of s_i where $1 \leq i \leq n$ and $1 \leq j \leq |s_i|$. Assume that $c < s_i$ for $1 \leq i \leq n$, that is, c is a common subsequence for S . We call every character of c , say $c[j]$, a *common character* of all sequences in S , i.e. $c[j] \in s_i$ for $1 \leq i \leq n$ and $1 \leq j \leq |c|$. Further, for each pair of two common characters $c[j_1]$ and $c[j_2]$, $1 \leq j_1 < j_2 \leq |c|$ (assume that $|c| \geq 2$), there must exist two positions u_i and v_i in s_i such that $s_i[u_i] = c[j_1]$ and $s_i[v_i] = c[j_2]$ where $1 \leq u_i < v_i \leq |s_i|$ for all $s_i \in S$ ($1 \leq i \leq n$). To obtain a good approximate LCS solution, we need to find out *which* common characters at *what* positions in the sequences that are able to constitute a common subsequence of S as long as possible. Our main idea in designing an

elegant ACO algorithm for LCS is thus to explore those promising common characters together with the information of their positions.

To effectively realize such an ACO algorithm for LCS, we have to consider: (1) how to transform the LCS problem into a state-based structure from which our artificial ants can build up solutions, (2) the state transition rule based upon which an ant moves from one state to another, (3) the pheromone updating rule that accommodates the pheromone trails to the learning experience on solution constructions of the colony, (4) the policy of local search to improve the solution quality and (5) the termination criterion for ACO to report a satisfactory solution. These designing issues will be discussed accordingly in the following subsections.

3.1. Problem transformation

Assume that the input of the LCS problem is a set of n sequences over Σ , $S = \{s_1, s_2, \dots, s_n\}$. Let m denote the number of the artificial ants we adopted in the ACO algorithm where $m < n$. In the very beginning, the algorithm randomly chooses a set of m sequences, referred to as R , from S ($R \subseteq S$) as the target set. Without losing generality, let $R = \{s_1, s_2, \dots, s_m\}$. (This can be done easily by renaming all sequences.) One particular artificial ant, say ant a_r , is assigned to sequence $s_r \in R$ ($1 \leq r \leq m$). a_r constructs its own solution by traversing s_r ; or more precisely, a_r searches for “good” common characters at their specific positions in s_r .

Let v_r be a position in s_r and v be the character at v_r in s_r , i.e. $v = s_r[v_r]$ ($\in \Sigma$), where $1 \leq v_r \leq |s_r|$ and $s_r \in R$. We regard each position v_r as a *state* of s_r in our ACO representation. Note that a state specifies a position and the character at this position simultaneously, also denoted by (v, v_r) if we need to express v and v_r explicitly (called *char-pos* for short, to emphasize their indivisible relationship). Each state, say v_r , in $s_r \in R$ is associated with a pheromone value, denoted by $\tau(v_r)$, to keep track of the merit of character v at position v_r for being a member of the solution(s) found by a_r as time passes. Initially, $\tau(v_r) = \tau_0$ and it will be updated by the pheromone updating rule (described later). Note that target set R is composed by m randomly selected sequence from S ($m < n = |S|$) so that only each state of $s_r \in R$ (for all r 's, $1 \leq r \leq m$) is associated with a pheromone value (while those characters in $s_{r'} \in S - R$ are not).

Let us re-examine the example in Section 2.1 where $S = \{s_1, s_2, s_3\}$ and $s_1 = \text{babacd}$, $s_2 = \text{acdcabc}$, $s_3 = \text{adcbac}$. We may set $m = 1$ and $R = \{s_1\}$. Then, the states with respect to s_1 are 1, 2, 3, 4, 5 and 6; or more specifically, (b, 1), (a, 2), (b, 3), (a, 4), (c, 5) and (d, 6) (which are all states/char-pos's for this problem under $R = \{s_1\}$). Each of them is associated with a pheromone value. Ant a_1 is assigned to s_1 for constructing solutions. Some states may be good candidates of common char-pos's because they may build up longer common subsequences, while others may not. In this example, state (b, 1) ($b = s_1[1]$) is not good enough since the LCS that it leads to is bc . Likewise, (a, 4) and (d, 6) are not either. On the other hand, states (a, 2), (b, 3) and (c, 5) are good states because they together constitute abc which is the LCS of S . Ant a_1 is expected to find out these good states in s_1 in the ACO algorithm.

A solution to the LCS problem obtained by ant a_r can thus be described as a series of feasible states in $s_r \in R$, just like (a, 2), (b, 3) and (c, 5) in s_1 in the example above. Note that to ensure states (a, 2), (b, 3) and (c, 5) in s_1 constitute an LCS of S (or to be feasible), a, b and c should also be common characters of s_2 and s_3 in the same order. In fact, subsequence abc can be found in s_2 at positions 1, 5, 6 and in s_3 at positions 1, 4, 5. The more states the ant traverses, the longer the common subsequence it constructs. The decision about which state to move in s_r for ant a_r depends upon the state transition rule defined in the next subsection.

3.2. State transition rule

The state transition rule consists of two components: (1) the *pheromone intensity* and (2) the *heuristic value*, with respect to each state in $s_r \in R$. For each state v_r in s_r , a pheromone value $\tau(v_r)$ is maintained to reflect the learning experience of the previous ants about choosing character v at position v_r to be a member of solution(s) as time passes. The heuristic value of state v_r , denoted by $\eta(v_r)$, for a_r in s_r provides a dynamic measurement (referred to as *visibility*) about the preference of moving from the current state to state v_r in a local point of view.

Before introducing the heuristic function and state transition rule, we designate some terms. We call it a *step* of an ant when the ant moves from the current state to the next state according to the state transition rule, an *iteration* of an ant when the ant constructs one feasible solution, and a *generation* of the colony when all of the ants in the colony finish their own iterations.

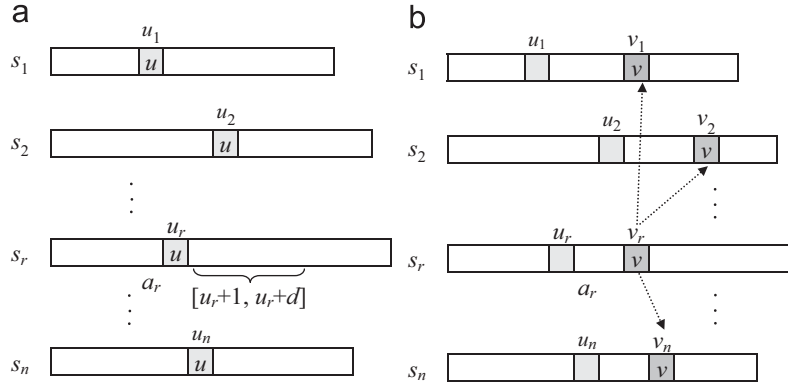


Fig. 5. Determining the l th common character ($c[l]$) of the solution in a certain step for ant a_r in s_r : (a) the current state u_r , (b) a possible next state v_r .

Suppose that ant a_r is now at state u_r in $s_r \in R$ and $u = s_r[u_r]$. Let c be the partially constructed solution (common subsequence) and currently $|c| = l - 1$ (i.e. $c = c[1]c[2] \dots c[l - 1]$). Let $u_1, u_2, \dots, u_n$ denote the positions of the $(l - 1)$ th common character in $s_1, s_2, \dots, s_n$, respectively, i.e. $s_1[u_1] = s_2[u_2] = \dots = s_n[u_n] = c[l - 1] = u$ (see Fig. 5(a)). In each step, a_r considers a range (called *window*, denoted by W) of d contiguous states, starting from positions $u_r + 1$ to $u_r + d$ (i.e. $W = [u_r + 1, u_r + d]$; strictly speaking, $W = [u_r + 1, \min(u_r + d, |s_r|)]$ for $0 \leq u_r \leq |s_r| - 1$), from which a_r either picks up one state to move (option 1) or skips them all (option 2) in this window according to the state transition rule described in the following. That is, in one step the ant may either choose one state among the d candidates in W and added the chosen character to c as $c[l]$, or just bypass all states in W .

Suppose that state $v_r \in W$ in s_r is chosen by a_r where $v = s_r[v_r]$. At the same time, before setting $c[l]$ to be v , we have to ensure that v is indeed a common character of all the other sequences s_k 's in S after position u_k 's for $1 \leq k \neq r \leq n$. We accomplish this in a greedy manner as follows. We search for positions $v_1, v_2, \dots, v_n$ at $s_1, s_2, \dots, s_n$, respectively, such that

$$s_k[v_k] = v \quad (1)$$

on condition that

$$s_k[v'_k] \neq v,$$

where $u_k + 1 \leq v'_k < v_k \leq |s_k|$ for all $1 \leq k \neq r \leq n$ (see Fig. 5(b)). That is, v_k is the first position after u_k in s_k such that $s_k[v_k] = v$ for $1 \leq k \neq r \leq n$. Note that v_r in s_r is chosen from window W according to the state transition rule by a_r , whereas v_k 's in s_k 's are determined based upon the chosen $v (=s_r[v_r])$ for $1 \leq k \neq r \leq n$.

Once a_r chooses state v_r to move in s_r , the characters between positions u_r and v_r cannot be used in further solution extensions because they will never be considered as the constituents to the current solution. We call them *not-chosen* characters in s_r . Intuitively, we treat it as a disadvantage in constructing a longer common subsequence if the total number of the not-chosen characters between u_i 's and v_i 's is large for all i 's, $1 \leq i \leq n$. Thus, our heuristic function in ACO is designed to evaluate the preference from the current state u_r to state v_r as follows:

$$\eta(v_r) = \frac{1}{\sum_{1 \leq i \leq n} (v_i - u_i)} \quad (2)$$

for each $v_r \in W$ ($1 < u_r + 1 \leq v_r \leq u_r + d \leq |s_r|$) where $1 \leq r \leq m$ subject to $s_i[v_i] = v$ and $s_i[u_i] = u$ for all $1 \leq i \leq n$. We prefer a larger heuristic value in ACO. That means our ant tends to choose a certain next state in W to move that results in a smaller number of total not-chosen characters. Note that this is simply a greedy and local concern.

Based upon the above discussions, the state transition rule for a_r at u_r in s_r to choose v_r to move or to skip all candidates in window W of size d is defined by the probabilistic function:

$$p(v_r) = \begin{cases} 1 & \text{if } q \leq q_0 \text{ and } v_r = \arg \max_{z_r \in W} \{\tau(z_r)\eta(z_r)^\beta\} \text{ (exploitation),} \\ \frac{\tau(v_r)\eta(v_r)^\beta}{\sum_{z_r \in W} \tau(z_r)\eta(z_r)^\beta} & \text{if } q_0 < q \leq q_1 \text{ (biased exploration),} \\ 0 & \text{otherwise } (q > q_1) \text{ (exclusion),} \end{cases} \quad (3)$$

where (a) $\tau(v_r)$ is the pheromone intensity at state v_r in which $v_r \in W (= [u_r + 1, \min(u_r + d, |s_r|)])$ in which $0 \leq u_r \leq |s_r| - 1$; (b) $\eta(v_r)$ is the heuristic value at state v_r defined by formula (2); (c) $\beta > 0$ is a parameter adjusting the relative influence between $\tau(v_r)$ and $\eta(v_r)$; (d) q_0 and q_1 , $0 \leq q_0 < q_1 \leq 1$, are parameters controlling the relative favorites for the searching strategies of *exploitation*, *biased exploration* and *exclusion*; and (e) q is a random number, $0 \leq q \leq 1$. With regard to the exploitation, a_r chooses v_r , which maximizes $\tau(v_r)\eta(v_r)^\beta$ among all candidates in W , to move. For the biased exploration, a_r chooses v_r to move depending on a biased probability proportional to $\tau(v_r)\eta(v_r)^\beta$ as indicated in formula (3). Regarding the exclusion, a_r does not choose anything from W .

In the cases of exploitation or biased exploration, a state v_r with a higher pheromone intensity and a larger heuristic value is preferred. A higher pheromone intensity for v_r implies that v_r has frequently been chosen as one of the constituents to the good solutions in previous iterations (or simply v_r possesses more historical learning experience). A larger heuristic value for v_r indicates that choosing v_r results in a smaller number of the total not-chosen characters as compared to other states in W in this step (which is a greedy heuristic based upon a local situation). Thus, the ants tend to choose those states that have been frequently chosen by the previous ants which could find good solutions; and those that prevent the total number of not-chosen characters in the current step from growing large. In essence, the historical experience retained by pheromone trails and the local information from the heuristic estimation are expected to guide the ants to establish a reinforcement learning mechanism to search for good solutions.

To keep track of the states in all sequences for solution c constructed so far, we should memorize positions $v_1, v_2, \dots, v_n$ in $s_1, s_2, \dots, s_n$, respectively. We call the n -tuple

$$V_r^l = \langle v_1, v_2, \dots, v_n \rangle \quad (4)$$

the position vector of the l th common character chosen by a_r in $s_r \in R$ for $1 \leq l \leq |c|$. It is easy to see,

$$V_r^{l-1} = \langle u_1, u_2, \dots, u_n \rangle;$$

and in the beginning of each iteration,

$$V_r^0 = \langle 0, 0, \dots, 0 \rangle$$

for all $s_r \in R$. For the example in Section 3.1, the construction of solution $c = abc$ implies $V_1^0 = \langle 0, 0, 0 \rangle$, $V_1^1 = \langle 2, 1, 1 \rangle$, $V_1^2 = \langle 3, 5, 4 \rangle$ and $V_1^3 = \langle 5, 6, 5 \rangle$ with respect to $R = \{s_1\}$.

Before the next step starts, the values of $u_1, u_2, \dots, u_n$ are replaced by $v_1, v_2, \dots, v_n$, respectively. Further, the starting position of W should be updated as $v_r + 1 (= u_r + 1)$. If $u_r > |s_r| - 1$ or there exists some string s_k , $1 \leq k \neq r \leq n$, such that no such v_k can be found in formula (1), the current iteration ends and solution $c[1]c[2] \dots c[l-1]$ constructed so far is reported as the solution of a_r for this iteration.

In the case of exclusion, we simply set $u_r = u_r + d$ (based upon which W is consequently updated) for the next step. There is no need to update u_k 's for $1 \leq k \neq r \leq n$. Obviously, if $u_r > |s_r|$, the current iteration stops.

By adjusting the window's staring position from the beginning ($u_r = 0$) to the end ($u_r = |s_r| - 1$) step by step in s_r , ant a_r can construct a feasible common subsequence of S described by a number of states in an iteration. When each of the m ants in the colony finishes its own iteration in one generation, ACO adopts a process of pheromone update described in the next subsection to modify the pheromone intensities.

3.3. Pheromone updating rule

At the end of each generation, the ACO algorithm performs the *global pheromone update* for all states t_r 's, in all sequences s_r 's ($s_r \in R$) according to the rule

$$\tau(t_r) = (1 - \rho)\tau(t_r) + \rho\Delta\tau, \quad (5)$$

and

$$\Delta\tau = \begin{cases} \frac{|c^+|}{M} & \text{if } t_r \text{ is chosen by } a_r \text{ as a constituent state of } c^+, \\ 0 & \text{otherwise,} \end{cases}$$

where (a) ρ is the *evaporation* coefficient which simulates the natural phenomenon of pheromone evaporation in real ant colonies; (b) c^+ denotes the *generation-best* solution which is the best one among the m solutions found by the m ants in the colony during the current generation; and (c) M is the length of the LCS of a random selected pair of sequences (s_i, s_j) , i.e. $M = |LCS(s_i, s_j)|$ where $s_i, s_j \in S$. The added amount $|c^+|/M$ is designed to be in the range of $(0, 1)$ in order to prevent the accumulated amount of pheromone intensity from growing unboundedly. Basically, our algorithm evaporates (decreases) the pheromone intensities of all states in all sequences s_r 's in R , and only rewards (increases) the pheromone intensities of those states in each of s_r 's that are the constituents of the generation-best solution c^+ by adding some amount of pheromone proportional to the solution quality of c^+ (i.e. $|c^+|/M$). Thus, those states in s_r 's that are chosen to be members of c^+ will accumulate more pheromone intensities. Note that the effect of evaporation makes the pheromone intensities of all states in s_r 's decrease as time goes by so that the bad decisions ever made could be forgotten over time. It is noted again that only those states in $s_r \in R$ (for all r 's, $1 \leq r \leq m$) are associated with pheromone trails (while those characters in $s_{r'} \in S - R$ are not).

The accumulated pheromone intensities on those states that have been frequently chosen as the constituents of the previous generation-best solutions will attract the ants in the following iterations to choose them with higher probabilities. The searching ability of ants is capable to evolve by way of the indirect communications of pheromone trails over generations.

3.4. Local search

In order to further improve the generation-best solution c^+ , the ACO algorithm conducts a local search procedure after each iteration. Assume that c^+ is found by a_r in $s_r \in R$ and the length of c^+ is l^+ . From formula (4), we know that the positions of the l th common character ($c^+[l]$) in $s_1, s_2, \dots, s_n$ are kept in $V_r^l = \langle v_1^l, v_2^l, \dots, v_n^l \rangle$ for a_r (see Fig. 5(b)) where $1 \leq l \leq l^+$ and $V_r^0 = \langle 0, 0, \dots, 0 \rangle$. We specify one additional boundary condition: $V_r^{l^++1} = \langle |s_1| + 1, |s_2| + 1, \dots, |s_n| + 1 \rangle$ and we set $c^+[0]$ and $c^+[l^+ + 1]$ to be null characters. It is easy to see that the positions of $c^+[0], c^+[1], \dots, c^+[l^+], c^+[l^+ + 1]$ in s_r are recorded by $v_r^0, v_r^1, \dots, v_r^{l^+}, v_r^{l^++1}$, respectively, where v_r^j is the r th coordinate in vector V_r^j for $1 \leq r \leq m$ and $0 \leq j \leq (l^+ + 1)$. We call (v_r^{l-1}, v_r^l) the l th *interval* of s_r with respect to c^+ for $1 \leq l \leq (l^+ + 1)$ and there are totally $|c^+| + 1$ intervals in s_r .

The local search proceeds as follows. At first, one sequence, namely s_y , is randomly chosen from R , $1 \leq y \leq m$. In the l th interval (v_y^{l-1}, v_y^l) of s_y , we consider h states which are with the highest h pheromone intensities in the interval for $1 \leq l \leq (l^+ + 1)$ where $v_y^0 = 0$, $v_y^{l^++1} = |s_y| + 1$ and h is a adjustable parameter, $1 \leq h < (v_y^l - v_y^{l-1})$. Let $e_y^1, e_y^2, \dots, e_y^h$ be the h states in s_y where $v_y^{l-1} < e_y^i < e_y^j < v_y^l$ for $1 \leq i < j \leq h$. Each of the h candidate states, say e_y^i , character $e^i (=s_y[e_y^i])$ will be inserted into c^+ (between $c^+[l-1]$ and $c^+[l]$) only when the following two conditions are met:

- (1) There exists position x_k in s_k such that $s_k[x_k] = e^i$ subject to $s_k[x'_k] \neq e^i$ for $v_k^{l-1} < x'_k < x_k \leq v_k^{l-1} + \delta$ in which $\delta = g \times |s_y|/l^+$; and
- (2) $c^+[l:l^+] < s_k[x_k + 1:|s_k|]$;

for $1 \leq k \neq y \leq m$, $1 \leq i \leq h$ and $1 \leq l \leq (l^+ + 1)$ where $1 \leq y \leq m$, $g \geq 1$ is a parameter specifying the maximum range $\delta (=g \times |s_y|/l^+)$ within which an insertion is only considered for each of $s_k \in S$ ($k \neq y$), and $s[a:b]$ denotes the

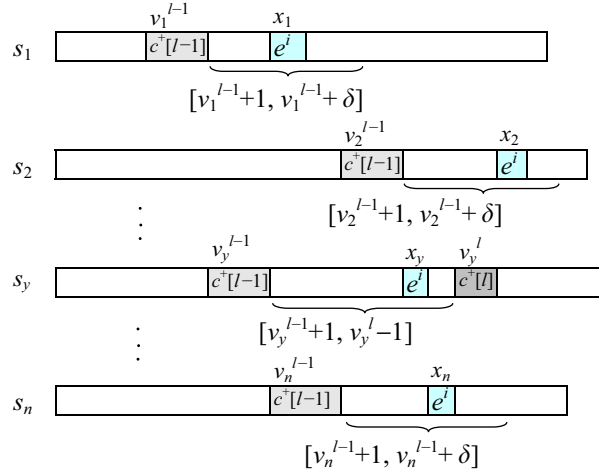


Fig. 6. Inserting e^i between $c^+[l-1]$ and $c^+[l]$ (only when conditions (1) and (2) are met).

substring of s from position a to position b . Note that if any of the two conditions is not satisfied, we simply ignore candidate state e_y^i .

Condition (1) specifies that the position of the first occurrence of e^i (at position x_k) in s_k after v_k^{l-1} must be in the range $x_k \in [v_k^{l-1} + 1, v_k^{l-1} + \delta]$ for $1 \leq k \neq y \leq n$ (in s_y , $x_y = e_y^i \in [v_y^{l-1} + 1, v_y^l - 1]$). Fig. 6 shows a legal insertion of e^i between $c^+[l-1]$ and $c^+[l]$ where $x_k \in [v_k^{l-1} + 1, v_k^{l-1} + \delta]$ for $1 \leq k \neq y \leq n$ and $x_y = e_y^i \in [v_y^{l-1} + 1, v_y^l - 1]$. This is also a greedy approach with a local concern. The characters between positions v_k^{l-1} and x_k in s_k are *not-chosen characters of local search* (since they will not be considered in further local searches) for $1 \leq k \neq y \leq n$. Thus, we regard it as a disadvantage if any insertion results in a large number (more than δ) of not-chosen characters in local search for any sequence s_k , $1 \leq k \neq y \leq n$. Note that any insertion of $e^i (=s_y[e_y^i])$ into s_k ($k \neq y$) after position v_k^{l-1} may change v_k^l (the original position of the $c^+[l]$ in s_k); but, v_y^l (the original position of the $c^+[l]$ in s_y) would never be changed since an insertion is only possible within the l th interval (v_y^{l-1}, v_y^l) of s_y and $s_y[v_y^l](=c^+[l])$ remains as well a member of c^+ (see Fig. 6).

Condition (2) states that $c^+[l:l^+]$ should still be a common subsequence of substring $s_k[x_k+1:|s_k|]$ for $1 \leq k \neq y \leq n$. This implies that all of the common characters in c^+ ought to be preserved in all s_k 's during the local search. We prefer to preserve all those common characters in c^+ (even though their positions in all $s_k \in S$ ($s_k \neq s_y$) might be changed due to some possible insertions) for the reason that c^+ was found by a colony of experienced (as generations pass) ants in the current generation.

In summary, for each interval (v_y^{l-1}, v_y^l) of $s_y \in R$, our local search tries to insert some potential characters (those with h highest pheromone intensities within the interval) into the current c^+ without incurring more than δ not-chosen characters in each sequence s_k ($s_k \in S, s_k \neq s_y$) and without impairing any common character in c^+ . Since s_y is randomly chosen each time ACO applies a local search, the insertion attempts based upon the intervals of s_y can be operated with an equal probability in each of the sequences in R over generations.

3.5. Termination criterion

In the following experiments, the ACO algorithm stops when the ants cannot improve the generation-best solution any more within 100 consecutive generations. This indicates a situation where further improvement becomes less likely.

4. Experimental results and discussions

The test platform in our experiments was a personal computer running Red Head Linux with an AMD Athlon 2100+ CPU and 512 MB RAM. The ACO algorithm was implemented in C++. The biological sequence sets tested consist

Table 1
Default values for the ACO parameters

Parameter	q_0	q_1	β	ρ	d	m	τ_0	g	h
Value	0.9	0.95	2	0.1	10 (Σ_{DNA}) or 24 (Σ_{protein})	16	1	10	5

Table 2
Comparison of ACO_{NoLS} and ACO_{LS}

n	DNA				Protein			
	Solution value		CPU time (s)		Solution value		CPU time (s)	
	ACO _{NoLS}	ACO _{LS}	ACO _{NoLS}	ACO _{LS}	ACO _{NoLS}	ACO _{LS}	ACO _{NoLS}	ACO _{LS}
40	138.4	142.6	9.6	18.0	42.2	43.9	8.7	14.5
60	137.5	141.8	12.2	19.5	41.0	42.8	13.0	21.9
80	123.2	129.7	13.8	27.4	37.6	39.6	13.6	24.6
100	118.8	124.8	15.5	31.2	34.8	36.8	15.7	33.0

of two categories of alphabets:

- (1) Σ_{DNA} , the alphabet of DNA sequences with $|\Sigma_{\text{DNA}}| = 4$; and
- (2) Σ_{protein} , the alphabet of protein sequences with $|\Sigma_{\text{protein}}| = 20$.

First of all, we conducted extensive preliminary experiments in order to determine the values of the parameters in the ACO algorithm. Based upon the outcomes of these experiments, the values of the parameters are summarized in Table 1.

These values trigger the ACO algorithm to produce satisfactory solutions to LCS in most of the test instances in our preliminary experiments. The empirical study about how different parameter values influence the performances of ACO was discussed in [37]. Note that the window size d was set as 10 for Σ_{DNA} or 24 for Σ_{protein} , and parameters g and h were adopted in the local search procedure.

Then, we exhibit the effect of local search in ACO. We randomly selected DNA and protein sequences from National Center for Biotechnology Information (NCBI) for $n = 40, 60, 80$ and 100. The length of each sequence was set to be 600, that is, we kept the first 600 characters and truncated the rest for each of the randomly selected sequence. Table 2 shows the computational results including the solution values (the length of common subsequence) and execution time of these test sets *with* or *without* the local search procedure in ACO, referred to as ACO_{LS} and ACO_{NoLS}, respectively. Each reported value in Table 2 is the average result of 10 independent runs.

It is easily seen from Table 2 that ACO_{LS} obtains better solution values than ACO_{NoLS} for both DNA and protein sequences. The contribution of local search in improving the solution quality of ACO is quite positive. Even though the execution time needed by ACO_{LS} is more than that by ACO_{NoLS}, the time spent by ACO_{LS} is acceptable. For a particular DNA case, say $n = 100$, ACO_{LS} reported an average solution value of 124.8 which is longer than that by ACO_{NoLS} (118.8), and it took 31.2 s on average, a rather acceptable time. Therefore, we adopted the local search procedure in ACO for all of the following experiments. That is, whenever we mention the results of ACO, we actually mean the results of ACO_{LS}.

To better realize the performance of the ACO algorithm in finding LCS for multiple biological sequences, we implemented ExpA [21] and BNMAS [22] as well in C++ deliberately for the purpose of comparing. The goals of the experiments were to explore the following questions:

- (1) What are the performances of ExpA, BNMAS and ACO when dealing with DNA sequences?
- (2) What are their performances when dealing with protein sequences?
- (3) What are their performances when the size of the test set becomes large?

Table 3

Solution values by ExpA, BNAMS and ACO for sequence sets from Ψ_{DNA}

n	Random			Rat			Virus		
	ExpA	BNMAS	ACO	ExpA	BNMAS	ACO	ExpA	BNMAS	ACO
10	173	169.2	197.2 (2.0)	160	157.5	182.0 (2.4)	174	176	197.6 (1.3)
15	164	160.7	185.2 (1.3)	147	146.8	166.6 (1.3)	161	162	183.6 (1.3)
20	159	155.4	176.2 (1.3)	147	143.3	160.0 (1.3)	154	156	173.8 (2.5)
25	154	151.1	172.2 (0.7)	143	138.7	155.8 (1.3)	162	169	179.0 (1.8)
40	148	144.6	161.4 (1.3)	131	130.7	143.4 (0.8)	146	147	155.0 (2.1)
60	142	142.1	155.4 (1.3)	132	129.6	142.4 (1.7)	139	145	150.6 (1.3)
80	142	141.5	151.6 (0.8)	115	117.8	128.8 (0.7)	137	139	145.8 (1.3)
100	141	139.8	148.8 (1.3)	111	113.7	124.6 (2.0)	135	136	143.4 (2.7)
150	136	136.2	143.4 (0.8)	101	109.8	115.6 (1.3)	134	135	141.6 (0.8)
200	134	134.8	141.0 (0.6)	103	107.2	114.6 (2.3)	134	133	140.6 (1.3)

Values in parentheses are the standard deviations of the corresponding solution values.

Table 4

CPU time (s) by ExpA, BNAMS and ACO for sequence sets from Ψ_{DNA}

n	Random			Rat			Virus		
	ExpA	BNMAS	ACO	ExpA	BNMAS	ACO	ExpA	BNMAS	ACO
10	88	1.1	10.7 (2.0)	40.8	1.0	7.4 (1.9)	81.4	1.1	3.7 (0.7)
15	66.8	1.5	15.7 (5.4)	26.5	1.5	10.5 (2.4)	51.5	1.7	7.9 (2.0)
20	60.2	2.1	11.4 (0.8)	14.1	1.9	12.5 (3.8)	26.4	2.1	20.4 (6.5)
25	49.0	2.6	15.4 (1.7)	20.3	2.5	15.9 (4.0)	51.7	2.9	18.3 (5.3)
40	60.7	4.3	23.8 (10.3)	33.9	3.7	21.0 (4.6)	50.2	4.1	20.5 (3.4)
60	87.2	6.5	24.7 (3.2)	67.5	5.8	26.2 (8.9)	80.0	6.3	30.8 (9.3)
80	169.5	8.4	32.5 (5.9)	131.2	7.5	29.9 (4.9)	158.4	8.0	45.5 (6.9)
100	306.5	10.4	43.6 (10.4)	247.0	8.7	48.8 (17.9)	289.1	10.2	23.8 (10.3)
150	907.8	15.3	57.2 (17.1)	795.5	12.3	35.0 (6.8)	1008.5	14.9	50.0 (21.3)
200	2111.9	20.5	59.1 (9.6)	1866.7	16.4	65.5 (14.0)	2381.0	19.6	65.6 (15.6)

Values in parentheses are the standard deviations of the corresponding CPU times.

We designed three experiments to answer the above three questions accordingly. In each of the three experiments, we consider three types of DNA/protein sequences: (a) sets of randomly generated sequences (Random); (b) sets of real rats' sequences (Rat); and (c) sets of real viruses' sequences (Virus). We summarize the computational results, particularly the solution values (the lengths of the common subsequence found) and the CPU time needed with respect to the test sets, in the following. It is noted that each value obtained from the BNAMS or ACO algorithms is the averaged result of 10 independent runs.

4.1. Experiment 1. Comparing performances of ExpA, BNAMS and ACO for DNA sequences

We randomly generated 200 DNA sequences as target set $\Psi_{\text{DNA-Random}}$ and randomly selected 200 real rats' and viruses' DNA sequences from NCBI as target sets $\Psi_{\text{DNA-Rat}}$ and $\Psi_{\text{DNA-Virus}}$, respectively. The length of each sequence was set to be 600 (i.e. the first 600 characters of each sequence from NCBI). To observe the results with respect to the sizes (from small to middle) of sequences, we set n to be 10, 15, 20, 25, 40, 60, 80, 100, 150 or 200. For a certain n , we randomly selected n sequences from $\Psi_{\text{DNA-Random}}$, $\Psi_{\text{DNA-Rat}}$ and $\Psi_{\text{DNA-Virus}}$, respectively, as the test sequence sets.

Table 3 summarizes the solution values found by ExpA and average solution values found by BNAMS and ACO, respectively, with respect to DNA sequences out of $\Psi_{\text{DNA-Random}}$, $\Psi_{\text{DNA-Rat}}$ and $\Psi_{\text{DNA-Virus}}$. Table 4 shows the CPU time in seconds needed by ExpA and the average CPU time needed by BNAMS and ACO for reporting the final solutions to the corresponding sequence sets. The standard deviations corresponding to the average solution values and

Table 5

Relative performance ratios of ACO over ExpA, ACO over BNMAS and BNMAS over ExpA

n	$((ACO - ExpA)/ExpA) \times 100\%$			$((ACO - BNMAS)/BNMAS) \times 100\%$			$((BNMAS - ExpA)/ExpA) \times 100\%$		
	Random	Rat	Virus	Random	Rat	Virus	Random	Rat	Virus
10	14.0	13.8	13.6	16.5	15.6	12.3	−2.20	−1.56	1.38
15	12.9	13.3	14.0	15.2	13.5	13.3	−2.01	−0.14	0.91
20	10.8	8.8	12.9	13.4	11.7	11.4	−2.26	−2.52	1.30
25	11.8	9.0	10.5	14.0	12.3	5.9	−1.88	−3.03	4.28
40	9.1	9.5	6.2	11.6	9.7	5.4	−2.30	−0.25	0.82
60	9.4	7.9	8.3	9.4	9.9	3.9	0.07	−1.82	3.98
80	6.8	12.0	6.4	7.1	9.3	4.9	−0.35	2.43	1.27
100	5.5	12.3	6.2	6.4	9.6	5.4	−0.85	2.46	0.84
150	5.4	14.5	5.7	5.3	5.3	4.9	0.15	8.71	0.75
200	5.2	11.3	4.9	4.6	6.9	5.7	0.60	4.08	−0.75

execution time by ACO are also shown in Tables 3 and 4, while those by BNAMS are omitted since they are less than 1 (for solution values) and 1 s (for execution time) for all of the test sets.

To ease the comparison for the solution qualities of these approaches, we compute the *relative performance ratios* of ACO over ExpA, ACO over BNMAS and BNMAS over ExpA in Table 5, which are evaluated by $((ACO - ExpA)/ExpA) \times 100\%$, $((ACO - BNMAS)/BNMAS) \times 100\%$ and $((BNMAS - ExpA)/ExpA) \times 100\%$, respectively, where *ACO*, *BNMAS* and *ExpA* denote the (average) solution values reported by ACO, BNMAS and ExpA accordingly.

It can be easily seen from Tables 3 and 5 that ACO delivers the best solution values among the three approaches in our test sets. The stability of the performance of ACO under different runs for a given DNA problem instance can be observed from the information of the standard deviations (obtained from 10 independent runs) in Table 3. As shown in Table 4, it is exciting to realize that the execution time needed by BNMAS or ACO is much less than ExpA. For a particular test set, say 200 sequences of rats, ACO took 65.6 s on average (with a standard deviation of 14.0) to report a solution with an average length of 114.6 (with a standard deviation of 2.3), BNAMS spent merely 16.4 s to obtain a solution with an average length of 107.2, while ExpA needed about 31 min (1866.7 s) to find a solution with a length of 103 only. BNMAS is the most computationally efficient algorithm among the three approaches. Although ACO needs more time than BNMAS, it delivers the best solution among the three and the time needed by ACO is still reasonable (especially when compared with that by ExpA). The effectiveness and efficiency of the ACO algorithm in finding the LCS for DNA sequences can be demonstrated in this experiment.

4.2. Experiment 2. Comparing performances of ExpA, BNMAS and ACO for protein sequences

As in Experiment 1, we randomly generated 200 protein sequences as target set $\Psi_{\text{protein-Random}}$ and randomly selected 200 real rats' and viruses' protein sequences from NCBI as target sets $\Psi_{\text{protein-Rat}}$ and $\Psi_{\text{protein-Virus}}$, respectively. The length of each sequence was set to be 600 (i.e. the first 600 characters of each sequence from NCBI) and we set n to be 10, 15, 20, 25, 40, 60, 80, 100, 150 or 200. For a specific n , we randomly selected n sequences from $\Psi_{\text{protein-Random}}$, $\Psi_{\text{protein-Rat}}$ and $\Psi_{\text{protein-Virus}}$ as the test sets.

Table 6 exhibits the computational results of the solution values by ExpA, BNMAS and ACO with respect to protein sequences from $\Psi_{\text{protein-Random}}$, $\Psi_{\text{protein-Rat}}$ and $\Psi_{\text{protein-Virus}}$. Table 7 lists the CPU time in seconds needed by these three approaches with respect to the corresponding sequence sets. Table 8 summarizes the relative performance ratios of ACO over ExpA, ACO over BNMAS and BNMAS over ExpA in terms of the solution quality. Note that the standard deviations of the average solution values (execution time) in BNMAS are less than 1 (1 s) for all of the test cases.

The results in Tables 6 and 8 reveal the effectiveness of ACO in obtaining superior LCS solutions as compared to ExpA and BNMAS for protein sequences. In all of our test cases, the lengths of common subsequences found by ACO are the longest among the three approaches. It is also noted that, in most of our test cases, BNMAS produces longer

Table 6

Solution values by ExpA, BNMAS and ACO for sequence sets from Ψ_{protein}

<i>n</i>	Random			Rat			Virus		
	ExpA	BNMAS	ACO	ExpA	BNMAS	ACO	ExpA	BNMAS	ACO
10	34	39.9	54.0 (1.1)	48	47.9	63.4 (1.3)	50	48.5	65.6 (0.8)
15	29	36.7	46.2 (1.6)	44	43.2	56.6 (0.8)	46	45.8	55.8 (1.3)
20	26	31.7	42.4 (1.3)	38	37.9	47.8 (0.7)	39	41.3	53.6 (1.3)
25	25	31.9	40.0 (1.1)	35	37.9	46.2 (1.3)	39	40.9	49.6 (0.8)
40	23	29.9	34.2 (0.7)	34	35.0	44.2 (1.3)	35	37.4	46.4 (0.8)
60	24	27.3	30.6 (0.8)	33	34.7	43.0 (0.4)	35	36.2	43.4 (0.8)
80	24	26.6	29.0 (1.1)	28	33.2	39.6 (0.8)	34	35.0	43.0 (0.4)
100	25	26.1	28.4 (0.8)	21	29.1	37.0 (1.1)	34	34.3	42.0 (1.1)
150	24	24.4	26.0 (0.4)	22	27.9	34.0 (1.1)	34	34.0	42.6 (0.8)
200	23	23.5	25.0 (0.2)	21	24.9	32.4 (1.3)	34	33.8	41.0 (0.2)

Values in parentheses are the standard deviations of the corresponding solution values.

Table 7

CPU time (s) by ExpA, BNMAS and ACO for sequence sets from Ψ_{protein}

<i>n</i>	Random			Rat			Virus		
	ExpA	BNMAS	ACO	ExpA	BNMAS	ACO	ExpA	BNMAS	ACO
10	0.6	3.6	7.4 (2.1)	0.7	4.1	9.2 (2.5)	0.8	3.8	3.5 (1.2)
15	1.6	5.1	9.3 (2.5)	1.5	5.5	8.9 (2.4)	1.7	5.2	10.4 (1.6)
20	3.9	6.0	11.4 (4.9)	3.6	6.1	14.6 (5.8)	4.5	6.9	10.8 (0.5)
25	6.9	8.1	10.5 (2.3)	6.4	8.3	11.5 (1.2)	7.1	8.4	13.2 (4.5)
40	24.3	12.4	14.1 (4.8)	23.4	11.4	14.6 (3.2)	26.2	12.0	17.1 (2.6)
60	70.7	17.3	17.3 (1.3)	73.5	18.8	31.5 (6.9)	88.5	18.1	27.7 (4.2)
80	165.1	21.7	22.9 (3.0)	186.8	23.9	32.4 (10.1)	208	23.4	38.1 (11.4)
100	322.0	27.0	25.6 (0.1)	343.3	26.2	42.4 (8.8)	417.3	27.3	23.4 (5.1)
150	1087.5	38.8	40.8 (7.4)	1145.7	34.3	49.1 (8.1)	1771.2	46.9	71.4 (19.8)
200	2531.2	53.3	55.4 (4.7)	2755.9	40.1	75.7 (17.0)	4187.8	54.9	78.9 (21.7)

Values in parentheses are the standard deviations of the corresponding CPU times.

Table 8

Relative performance ratios of ACO over ExpA, ACO over BNMAS and BNMAS over ExpA

<i>n</i>	$((ACO - ExpA)/ExpA) \times 100\%$			$((ACO - BNMAS)/BNMAS) \times 100\%$			$((BNMAS - ExpA)/ExpA) \times 100\%$		
	Random	Rat	Virus	Random	Rat	Virus	Random	Rat	Virus
10	58.8	32.1	31.2	35.3	32.4	35.3	17.3	−0.3	−3.1
15	59.3	28.6	21.3	25.9	31.0	21.8	26.4	−1.8	−0.4
20	63.1	25.8	37.4	33.8	26.1	29.8	22.1	−0.2	5.9
25	60.0	32.0	27.2	25.4	21.9	21.3	27.7	8.2	5.0
40	48.7	30.0	32.6	14.4	26.3	24.1	29.9	2.9	6.9
60	27.5	30.3	24.0	12.1	23.9	19.9	13.9	5.2	3.4
80	20.8	41.4	26.5	9.0	19.3	22.9	11.7	18.6	2.9
100	13.6	76.2	23.5	8.8	27.1	22.4	5.3	38.4	0.9
150	8.3	54.5	25.3	6.6	21.9	25.3	6.1	27.0	0.0
200	8.7	54.3	20.6	6.4	30.1	21.3	6.4	18.7	−0.6

common subsequences than ExpA. We may say that ACO is more capable to find longer common subsequences for protein sequences than BNMAS and ExpA. The results of the standard deviations also indicate that the ACO algorithm runs quite stable for protein sequences.

Table 9
Results of 300 sequences of Primates by ExpA, BNMAS and ACO

	Solution value			CPU time (s)			Relative performance ratio	
	ExpA	BNMAS	ACO	ExpA	BNMAS	ACO	$((ACO - ExpA) / ExpA) \times 100\%$	$((ACO - BNMAS) / BNMAS) \times 100\%$
DNA	104	105.6	108.2 (1.7)	6081.2	21.9	41.3 (9.8)	4.0	2.5
Protein	19	24.1	28.2 (0.7)	8438.7	37.1	45.6 (16.7)	48.4	17.0

Values in parentheses are the standard deviations of the corresponding results.

From the CPU time results in Table 7, we know that BNMAS took the least execution time among all, ACO needed more time than BNMAS, while ExpA spent much more time as compared to both BNMAS and ACO. Although ExpA can report a solution promptly when n is small, the poor solution quality is not attractive at all. As n increases, the execution time spent by ExpA grows rapidly. Nevertheless, the execution time needed by BNMAS or ACO remains reasonable even when n becomes larger. In particular, for the test set with 200 sequences of rats, ACO took 75.7 s on average (with a standard deviation of 17.0) to find an answer with an average length of 32.4 (with a standard deviation of 1.3), BNMAS took only 40.1 s to obtain an answer with an average length of 24.9, while ExpA spent about 45 min (2755.9 s) just to report an LCS with a length of 21. Although the time needed by ACO is more than that of BNMAS, it is quite acceptable.

The effectiveness and efficiency of ACO in dealing with the LCS problem for protein sequences is easily seen from these computational results. Moreover, by comparing the relative performance ratios of Tables 5 and 8, we have an interesting observation that the superiority of ACO over ExpA and BNMAS for protein sequences is even more significant than that for DNA sequences.

4.3. Experiment 3. Comparing performances of ExpA, BNMAS and ACO and for a large problem size

Now, we would like to apply the ExpA, BNMAS and ACO algorithms to solve LCS problems for DNA and protein sequences with a large problem size, namely $n = 300$, where each sequence is with a length of 600 as well. We randomly selected 300 DNA and protein sequences of Primates, respectively, from NCBI as the test sequence sets.

Table 9 summarizes the solution values and execution time by applying ExpA, BNMAS and ACO on these two test sets, as well as the relative performance ratios of ACO over ExpA and BNMAS, respectively. The standard deviations corresponding to the results of BNMAS are small so that they are omitted in Table 9.

For both DNA and protein sequences of Primates tested, the solution values obtained by ACO are again the best among the three approaches (whereas, those by BNMAS are better than those by ExpA). From the results of relative performance ratios in Table 9, we observe again that the superiority of ACO over ExpA and BNMAS for protein sequences is more appealing than that for DNA sequences. Likewise, BNMAS is the most efficient approach among all and ACO remains much more efficient than ExpA.

Based upon the above three experiments, we may say that the solution quality of ExpA is comparable to those of BNMAS and ACO only when the size of the alphabet is small (such as $|\Sigma_{\text{DNA}}| = 4$) and the problem size is large (say $n = 300$), but the execution time needed by ExpA is huge (as compared to that of BNMAS or ACO). When the alphabet size is large (such as $|\Sigma_{\text{protein}}| = 20$), ExpA is infeasible in terms of solution quality and computational efficiency. BNMAS is the most efficient among the three approaches. When solving a problem with a large number of sequences, BNMAS can report a better solution than ExpA in a quite short time. ACO takes more time than BNMAS, but the time needed is rather reasonable. The most attractive finding is that the solution quality obtained by ACO is better than those by ExpA and BNMAS for either DNA or protein sequences. Further, the effectiveness of ACO for protein sequences is more significant than that for DNA sequences.

5. Concluding remarks

In this paper, we successfully design an elegant ACO algorithm for the LCS problem. All of the parameters adopted in the proposed ACO algorithm were extensively tested to ensure that ACO can find solutions to the LCS problem

with a satisfactory quality. We compared the performance of our ACO algorithm against those of the known ExpA by Bonizzoni et al. and BNMAS by Huang et al. with respect to random and real DNA/protein sequences with various problem sizes. The computational results show that (1) in terms of solution quality ACO outperforms ExpA and BNMAS when dealing with both the DNA and protein sequences (especially the protein sequences), and (2) in terms of computational efficiency ACO and BNMAS are much better than ExpA for both DNA and protein sequences even when the problem size becomes large. Even though BNMAS is more efficient than ACO, the time needed by ACO is fairly acceptable. The proposed ACO algorithm is indeed promising to resolve the LCS problem of multiple biological sequences.

The proposed ACO algorithm for LCS has some interesting features as follows. The pheromone trails in the ACO algorithm are maintained on all characters of all sequences in R (a subset of the input set S where $|R| = m < |S| = n$ in which m is the number of ants in ACO). The heuristic function is designed to favor a certain state that leads to a smaller number of the total not-chosen characters within a pre-defined window for the current step (a local view). The decisions of the state transition rule include exploitation, biased exploration and exclusion in a pre-defined window (instead of the whole sequence). The local search procedure tries to insert more characters into the generation-best solution without impairing it. As far as we know, these features and skills are quite different from most of the approaches that can be found in the literature about the realization of ACO in combinatorial optimization. They are innovative and may be applied to solve other combinatorial problems, especially for those problems with character-based structures.

In our experiments, the suitable size of the window, which surely affects the computational efficiency, tends to be proportional to the size of the alphabet when sequence sets from different alphabets were tested. The relationship between them is worthy of further study so that ACO can be applied to find LCS for more types of sequences.

The accession numbers in NCBI with regard to the test sequences in this paper are available from http://tmue.edu.tw/~sjshyu/public/aco_lcs/accno-aco_lcs.html or http://www.csie.mcu.edu.tw/~sjshyu/resource/accno-aco_lcs.html.

Acknowledgements

The authors would like to express their immense gratitude to the anonymous referees for their valuable and constructive suggestions which improve the quality of the manuscript. Special thanks should also be sent to Mr. Chen, Kun, Mr. Kao, Chia Yang and Mr. Fang, Yuan Li for their assistance in the computational experiments.

References

- [1] Needleman SB, Wunsch CD. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology* 1970;48:443–53.
- [2] Wagner RA, Fischer MJ. The string-to-string correction problem. *Journal of the ACM* 1974;21(1):168–73.
- [3] Smith T, Waterman M. Identification of common molecular subsequences. *Journal of Molecular Biology* 1981;147:195–7.
- [4] Sankoff D, Kruskal JB, editors. Time warps, string edits, and macromolecules: the theory and practice of sequence comparison. Reading, MA: Addison-Wesley; 1983.
- [5] Miller W, Myers EW. A file comparison program. *Software—Practice & Experience* 1985;15(11):1025–40.
- [6] Baeza-Yates RA, Gonnet GH. A new approach to text searching. *Communications of the ACM* 1992;35:74–82.
- [7] Gusfield D. Algorithms on strings, trees and sequences. New York: Cambridge University Press; 1997.
- [8] Jones JC, Pevzner PA. An introduction to bioinformatics algorithms. Cambridge: The MIT Press; 2004.
- [9] Koller G, Raidl GR. An evolutionary algorithm for the maximum weight trace formulation of the multiple sequence alignment problem. *Lecture notes in computer science*, vol. 3242. Berlin: Springer; 2004. p. 302–11.
- [10] Shmygelka A, Aguirre-Hernandez R, Hoos H. An ant colony optimization algorithm for the 2D HP protein folding problem. *Lecture notes in computer science*, vol. 2463. Berlin: Springer; 2002. p. 40–52.
- [11] Hirschberg DS. Algorithms for the longest common sequence problem. *Journal of the ACM* 1977;24(4):664–75.
- [12] Masek W, Peterson M. A faster algorithm computing string edit distances. *Journal of Computer Systems and Science* 1980;20(1):18–31.
- [13] Apostolico A, Guerra C. The longest common subsequence problem revisited. *Algorithmica* 1987;18(1):315–36.
- [14] Breimer EA, Goldberg MK, Lim DT. A learning algorithm for the longest common subsequence problem. *Journal of Experimental Algorithms* 2003;8:1–12.
- [15] Bergroth L, Hakonen H, Raita T. A survey of longest common subsequence algorithms. *SPIRE'2000*; 2000. p. 39–48.
- [16] Maier D. The complexity of some problems on subsequences and supersequences. *Journal of the ACM* 1978;25(2):322–36.
- [17] Garey MR, Johnson DS. Computers and intractability: a guide to the theory of NP-completeness. New York: W.H. Freeman and Company; 1979.
- [18] Hakata K, Imai H. The longest common subsequence problem for small alphabet size between many strings. *Lecture notes in computer science*, vol. 650. Berlin: Springer; 1992. p. 469–78.

- [19] Irving RW, Fraser CB. Two algorithms for the longest common subsequence of three (or more) strings. *Lecture notes in computer science*, vol. 644. Berlin: Springer; 1992. p. 214–29.
- [20] Jiang T, Li M. On the approximation of shortest common supersequences and longest common subsequences. *SIAM Journal on Computing* 1995;24(5):1122–39.
- [21] Bonizzoni P, Vedova GD, Mauri G. Experimenting an approximation algorithm for the LCS. *Discrete Applied Mathematics* 2001;110:13–24.
- [22] Huang KS, Yang CB, Tseng KT. Fast algorithms for finding the common subsequence of multiple sequences. In: *Proceedings of international computer symposium*; 2004. p. 90–95.
- [23] In: Voss S, Martello S, Osman IH, Roucairol C, editors. *Meta-heuristics: theory and applications*. Norwell, MA: Kluwer Academic Publishers; 1996.
- [24] In: Corne D, Dorigo M, Glover F, editors. *New ideas in optimization*. New York: McGraw-Hill; 1999.
- [25] In: Ribeiro CC, Hansen P, editors. *Essays and surveys in metaheuristics*. Dordrecht: Kluwer Academic Publishers; 2001.
- [26] Dorigo M, Maniezzo V, Colomi A. Positive feedback as a search strategy. *Politecnico di Milano, Italy: Dipartimento di Elettronica*; 1991. p. 91–116.
- [27] Dorigo M. *Optimization, learning and natural algorithms*. PhD thesis, Politecnico di Milano, Italy; 1992.
- [28] Dorigo M, Gambardella LM. Ant colony system: a cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation* 1997;1(1):53–66.
- [29] Dorigo M, Gambardella LM. Ant colonies for the traveling salesman problem. *BioSystems* 1997;43:73–81.
- [30] Dorigo M, Caro GD, Gambardella LM. Ant algorithm for discrete optimization. *Artificial Life* 1999;5:137–72.
- [31] Maniezzo V, Colomi A. The ant system applied to the quadratic assignment problem. *IEEE Transactions on Knowledge and Data Engineering* 1999;11(5):769–78.
- [32] McMullen PR. An ant colony optimization approach to addressing a JIT sequencing problem with multiple objectives. *Artificial Intelligence in Engineering* 2001;15(3):309–17.
- [33] Merkle D, Middendorf M, Schmeck H. Ant colony optimization for resource-constrained project scheduling. *IEEE Transactions on Evolutionary Computing* 2002;6(4):333–46.
- [34] Shyu SJ, Yin PY, Lin BMT. An ant colony optimization algorithm for the minimum weight vertex cover problem. *Annals of Operations Research* 2004;131:283–304.
- [35] Shyu SJ, Lin BMT, Hsiao TS. Ant colony optimization for the cell assignment problem in PCS networks. *Computers & Operations Research* 2006;33(6):1713–40.
- [36] Page RDM, Holmes EC. *Molecular evolution: a phylogenetic approach*. Oxford: Blackwell Science; 1998.
- [37] Shyu SJ, Tsai CY. Empirical study on an ant colony optimization algorithm for the longest common subsequence problem. Technical Report 2007 [http://tmue.edu.tw/~sjshyu/public/aco_lcs/es_aco_lcs.pdf or http://www.csie.mcu.edu.tw/~sjshyu/resource/es_aco_lcs.pdf].